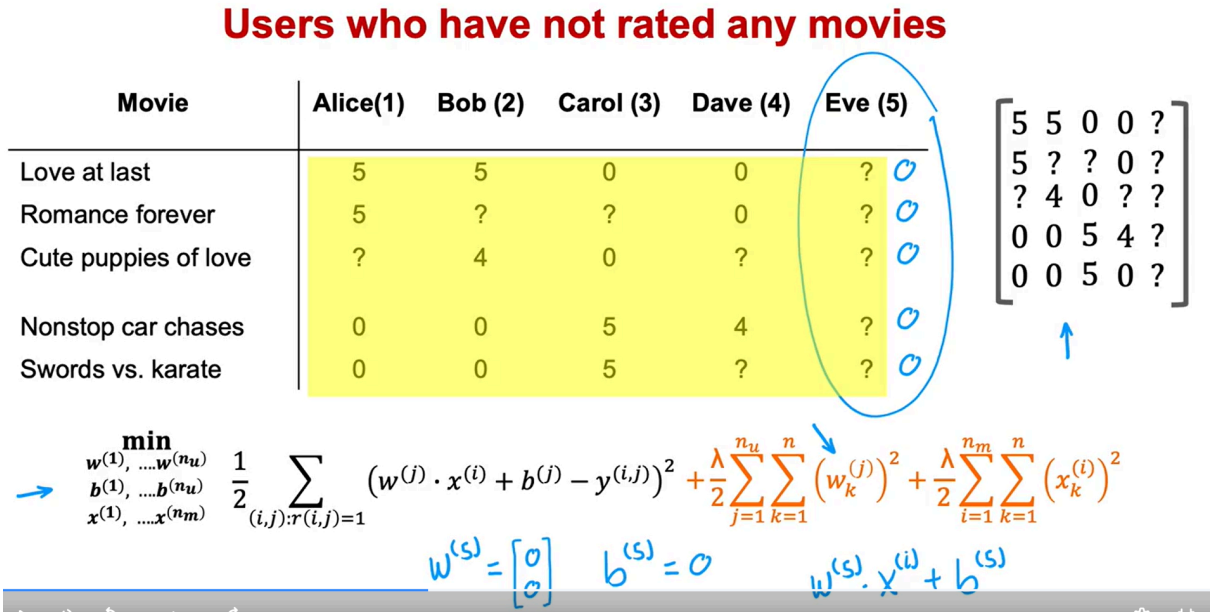
Recommender systems implementation detail

Mean normalization

In the case of building a recommended system with numbers wide such as movie ratings from one to five or zero to five stars, it turns out your algorithm will run more efficiently. And also perform a bit better if you first carry out mean normalization.



Back in the first course, you have seen how for linear regression, future normalization can help the algorithm run faster.

In the case of building a recommended system with numbers wide such as movie ratings from one to five or zero to five stars, it turns out your algorithm will run more efficiently. And also perform a bit better if you first carry out mean normalization. That is if you normalize the movie ratings to have a consistent average value, let's take a look at what that means. So here's the data set that we've been using.

And down below is the cost function you used to learn the parameters for the model. In order to explain mean normalization, I'm actually going to add fifth user Eve who has not yet rated any movies. And you see in a little bit that

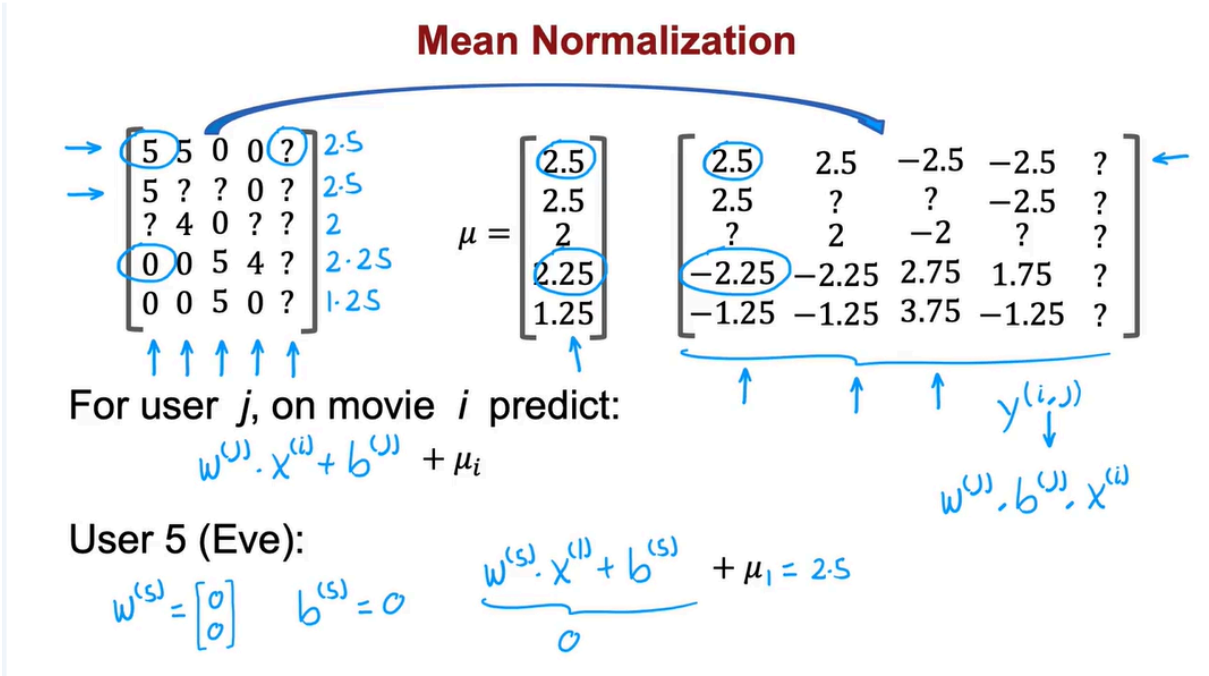
adding mean normalization will help the algorithm make better predictions on the user Eve.

In fact, if you were to train a collaborative filtering algorithm on this data, then because we are trying to make the parameters w small because of this regularization term. If you were to run the algorithm on this dataset, you actually end up with the parameters w for the fifth user, for the user Eve to be equal to [0 0] as well as quite likely b(5) = 0. Because Eve hasn't rated any movies yet, the parameters w and b don't affect this first term in the cost function because none of Eve's movie's rating play a role in this squared error cost function. And so minimizing this means making the parameters w as small as possible.

We didn't really regularize b. But if you initialize b to 0 as the default, you end up with b(5) = 0 as well. But if these are the parameters for user 5 that is for Eve, then what the average will end up doing is predict that all of Eve's movies ratings would be w(5) dot x for movie i + b(5). And this is equal to 0 if w and b above equals 0. And so this algorithm will predict that if you have a new user that has not yet rated anything, we think they'll rate all movies with zero stars and that's not particularly helpful.

So in this video, we'll see that mean normalization will help this algorithm come up with better predictions of the movie ratings for a new user that has not yet rated any movies. In order to describe mean normalization, let me take all of the values here including all the question marks for Eve and put them in a two dimensional matrix like this.

Just to write out all the ratings including the question marks in a more sustained and more compact way. To carry out mean normalization, what we're going to do is take all of these ratings and for each movie, compute the average rating that was given



So for example this movie rating was 5. I'm going to subtract 2.5 giving me 2.5 over here. This movie had a 0 star rating. I'm going to subtract 2.25 giving me a -2.25 rating and so on for all of the now five users including the new user Eve as well as for all five movies. Then these new values on the right become your new values of Y(i,j).

We're going to pretend that user 1 had given a 2.5 rating to movie one and the -2.25 rating to movie four. And using this, you can then learn w(j), b(j) and x(i) same as before for user j on movie i, you would predict w(j).x(i) + b(j). But because we had subtracted off μi for movie i during this mean normalization step, in order to predict not a negative star rating which is impossible for user rates from 0 to 5 stars. We have to add back this μi which is just the value we have subtracted out. So as a concrete example, if we look at what happens with user 5 with the new user Eve because she had not yet rated any movies, the average might learn parameters w(5) = [0 0] and say b(5) = 0. And so if we look at the predicted rating for movie one, we will predict that Eve will rate it w(5).x1 + b(5) but this is 0 and then + μ1 which is equal to 2.5.

So this seems more reasonable to think Eve is likely to rate this movie 2.5 rather than think Eve will rate all movie zero stars just because she hasn't rated any movies yet. And in fact the effect of this algorithm is it will cause the initial guesses for the new user Eve to be just equal to the mean of whatever other users have rated these five movies. And that seems more reasonable to take

the average rating of the movies rather than to guess that all the ratings by Eve will be zero.

It turns out that by normalizing the mean of the different movies ratings to be zero, the optimization algorithm for the recommender system will also run just a little bit faster. But it does make the algorithm behave much better for users who have rated no movies or very small numbers of movies. And the predictions will become more reasonable. In this example, what we did was normalize each of the rows of this matrix to have zero mean and we saw this helps when there's a new user that hasn't rated a lot of movies yet.

There's one other alternative that you could use which is to instead normalize the columns of this matrix to have zero mean. And that would be a reasonable thing to do too. But I think in this application, normalizing the rows so that you can give reasonable ratings for a new user seems more important than normalizing the columns. Normalizing the columns would hope if there was a brand new movie that no one has rated yet. But if there's a brand new movie that no one has rated yet, you probably shouldn't show that movie to too many users initially because you don't know that much about that movie.

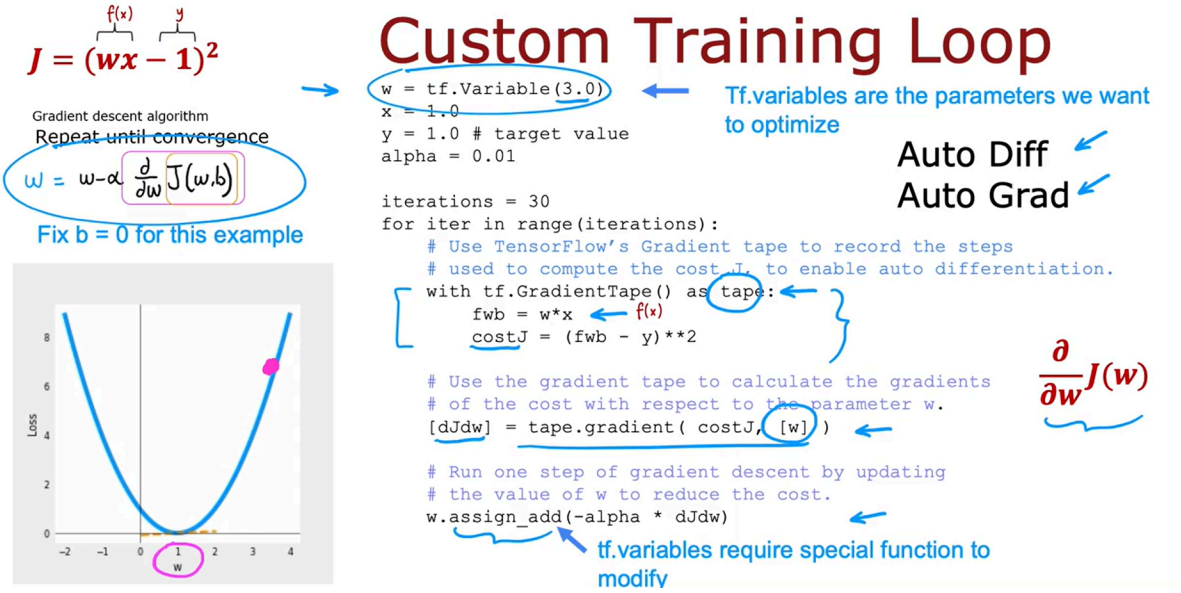
So normalizing columns the hope with the case of a movie with no ratings seems less important to me than normalizing the rules to hope with the case of a new user that's hardly rated any movies yet. And when you are building your own recommended system in this week's practice lab, normalizing just the roles should work fine. So that's mean normalization. It makes the algorithm run a little bit faster. But even more important, it makes the algorithm give much better, much more reasonable predictions when there are users that rated very few movies or even no movies at all. This implementation detail of mean normalization will make your recommended system work much better.

TensorFlow implementation of collaborative filtering

One of the reasons I like using TensorFlow for talks like these is that for many applications in order to implement gradient descent, you need to find the derivatives of the cost function, but TensorFlow can automatically figure out for you what are the derivatives of the cost function.

All you have to do is implement the cost function and without needing to know any calculus, without needing to take derivatives yourself, you can get

TensorFlow with just a few lines of code to compute that derivative term, that can be used to optimize the cost function. Let's take a look at how all this works.



I'm going to use a very simple cost function J=(wx-1) squared. So wx is our simplified $f_w(x)$ and y = 1.

And so this would be the cost function if we had f(x) = wx, y equals 1 for the one training example that we have, and if we were not optimizing this respect to b. So the gradient descent algorithm will repeat until convergence this update over here. It turns out that if you implement the cost function J over here, TensorFlow can automatically compute for you this derivative term and thereby get gradient descent to work.

I'll give you a high level overview of what this code does, w=tf.variable(3.0). Takes the parameter w and initializes it to the value of 3.0. Telling TensorFlow that w is a variable is how we tell it that w is a parameter that we want to optimize. I'm going to set x=1.0, y=1.0, and the learning rate alpha to be equal to 0.01. And let's run gradient descent for 30 iterations. So in this code will still do for iter in range iterations, so for 30 iterations.

And this is the syntax to get TensorFlow to automatically compute derivatives for you. TensorFlow has a feature called a gradient tape. And if you write this with tf our gradient tape as tape. This is compute f(x) as w*x and compute J as f(x)-y squared. Then by telling TensorFlow how to compute to costJ, and by doing it with the gradient taped syntax as follows, TensorFlow

will automatically record the sequence of steps. The sequence of operations needed to compute the costJ. And this is needed to enable automatic differentiation.

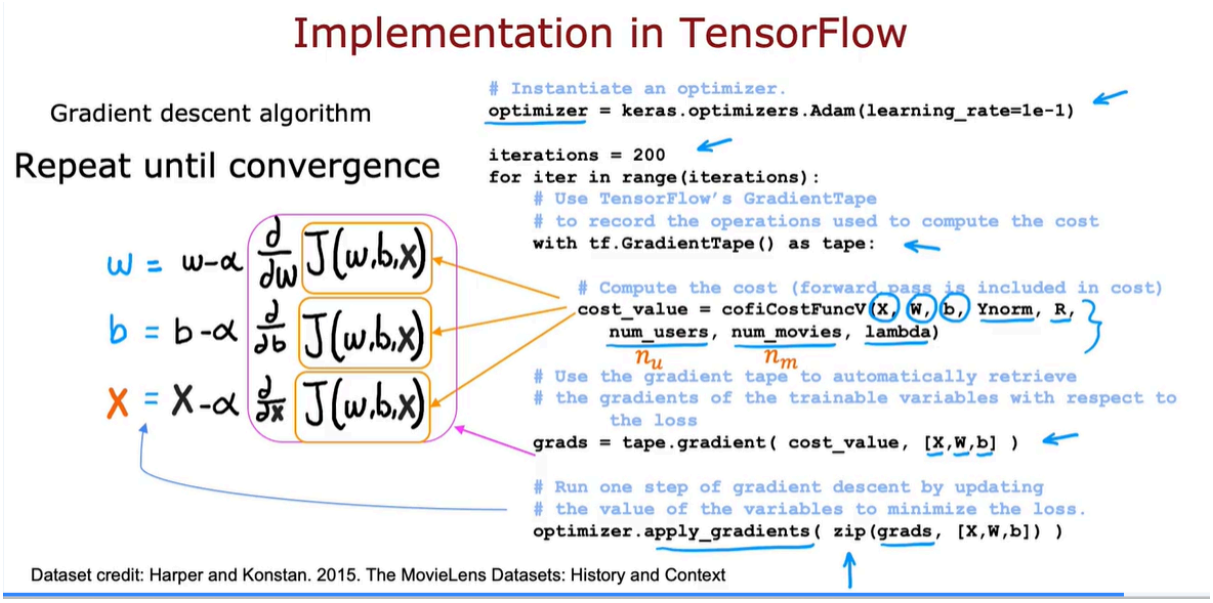
Next TensorFlow will have saved the sequence of operations in tape, in the gradient tape. And with this syntax, TensorFlow will automatically compute this derivative term, which I'm going to call dJdw. And TensorFlow knows you want to take the derivative respect to w. That w is the parameter you want to optimize because you had told it so up here. And because we're also specifying it down here. So now you compute the derivatives, finally you can carry out this update by taking w and subtracting from it the learning rate alpha times that derivative term that we just got from up above.

TensorFlow variables, tier variables requires special handling. Which is why instead of setting w to be w minus alpha times the derivative in the usual way, we use this assigned add function. But when you get to the practice lab, don't worry about it. We'll give you all the syntax you need in order to implement the collaborative filtering algorithm correctly. So notice that with the gradient tape feature of TensorFlow, the main work you need to do is to tell it how to compute the cost function J.

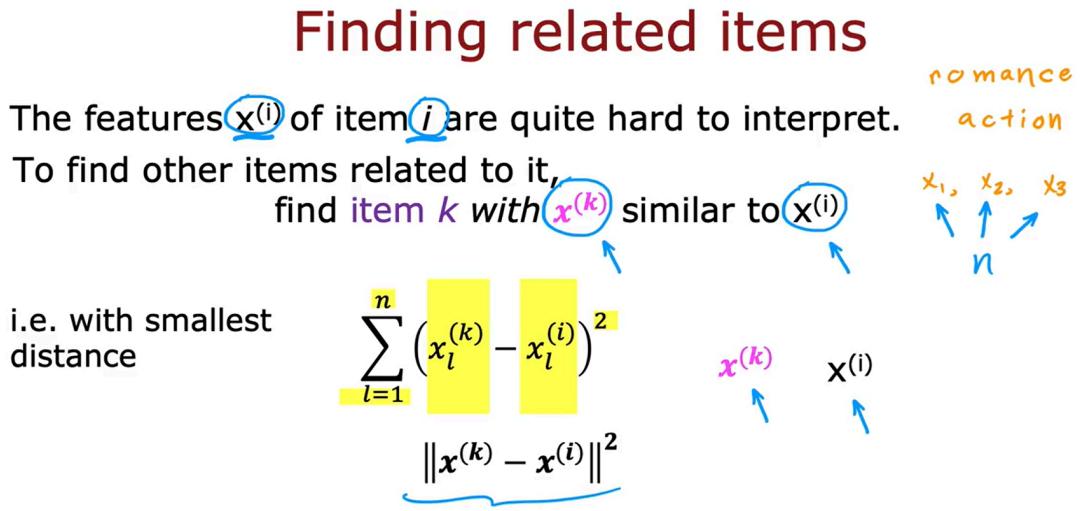
And the rest of the syntax causes TensorFlow to automatically figure out for you what is that derivative? And with this TensorFlow we'll start with finding the slope of this, at 3 shown by this dash line. Take a gradient step and update w and compute the derivative again and update w over and over until eventually it gets to the optimal value of w, which is at w equals 1.

So this procedure allows you to implement gradient descent without ever having to figure out yourself how to compute this derivative term. This is a very powerful feature of TensorFlow called Auto Diff. And some other machine learning packages like pytorch also support Auto Diff. Sometimes you hear people call this Auto Grad. The technically correct term is Auto Diff, and Auto Grad is actually the name of the specific software package for doing automatic differentiation, for taking derivatives automatically. But sometimes if you hear someone refer to Auto Grad, they're just referring to this same concept of automatically taking derivatives.

So let's take this and look at how you can implement to collaborative filtering algorithm using Auto Diff. And in fact, once you can compute derivatives automatically, you're not limited to just gradient descent. You can also use a more powerful optimization algorithm, like the adam optimization algorithm



Finding related items



n: number of features

Limitations of Collaborative Filtering

Cold start problem. How to

- rank new items that few users have rated?
- show something reasonable to new users who have rated few items?

Use side information about items or users:

- Item: Genre, movie stars, studio, ….
- User: Demographics (age, gender, location), expressed preferences, …